exercice

November 3, 2026

[1]:
```
%matplotlib inline
import os
import urllib.request
import matplotlib.pyplot as plt
import pandas as pd
import isoweek
```

Les données de l'incidence de la varicelle sont disponibles du site Web du Réseau Sentinelles. Nous les récupérons sous forme d'un fichier en format CSV dont chaque ligne correspond à une semaine de la période demandée. Nous téléchargeons le jeu de données complet, qui commence en 1990 et se termine avec une semaine récente.

Comme pour le syndrome grippal, nous travaillons sur une **copie locale** des données: le fichier n'est téléchargé que s'il n'existe pas encore. L'URL d'origine est conservée ici pour la traçabilité. Le code de la maladie dans l'URL est 7 pour la varicelle.

[2]:
```
data_url = "http://www.sentiweb.fr/datasets/incidence-PAY-7.csv"
data_file = "varicelle.csv"
```

Si le fichier local n'existe pas, nous téléchargeons les données depuis l'URL d'origine et nous les y déposons.

[3]:
```
if not os.path.exists(data_file):
    urllib.request.urlretrieve(data_url, data_file)
```

Voici l'explication des colonnes données sur le site d'origine:

Nom de colonne	Libellé de colonne
week	Semaine calendaire (ISO 8601)
indicator	Code de l'indicateur de surveillance
inc	Estimation de l'incidence de consultations en nombre de cas
inc_low	Estimation de la borne inférieure de l'IC95% du nombre de cas de consultation
inc_up	Estimation de la borne supérieure de l'IC95% du nombre de cas de consultation
inc100	Estimation du taux d'incidence du nombre de cas de consultation (en cas pour 100,000 habitants)
inc100_low	Estimation de la borne inférieure de l'IC95% du taux d'incidence du nombre de cas de consultation (en cas pour 100,000 habitants)

Nom de colonne	Libellé de colonne
inc100_up	Estimation de la borne supérieure de l'IC95% du taux d'incidence du nombre de cas de consultation (en cas pour 100,000 habitants)
geo_insee	Code de la zone géographique concernée (Code INSEE)
geo_name	Libellé de la zone géographique (ce libellé peut être modifié sans préavis)

La première ligne du fichier CSV est un commentaire, que nous ignorons en précisant `skiprows=1`.

Nous lisons ensuite le fichier CSV local. Un tiret `-` marque une valeur manquante: nous le déclarons avec `na_values`, sinon les colonnes numériques seraient lues comme du texte.

[5]:
```
raw_data = pd.read_csv(data_file, encoding='iso-8859-1', skiprows=1,␣
↪na_values=['-'])
raw_data
```

[5]:

	week	indicator	inc	inc_low	inc_up	inc100	inc100_low	\
0	202643	7	4736	2131	7341	7	3	
1	202642	7	2238	886	3590	3	1	
2	202641	7	2590	1247	3933	4	2	
3	202640	7	1918	812	3024	3	1	
4	202639	7	1224	234	2214	2	1	
5	202638	7	592	0	1329	1	0	
6	202637	7	862	43	1681	1	0	
7	202636	7	975	167	1783	1	0	
8	202635	7	1130	186	2074	2	1	
9	202634	7	1682	301	3063	2	0	
10	202633	7	1665	392	2938	2	0	
11	202632	7	4231	2124	6338	6	3	
12	202631	7	4064	1915	6213	6	3	
13	202630	7	6248	3767	8729	9	5	
14	202629	7	5719	3384	8054	8	5	
15	202628	7	8445	5252	11638	13	8	
16	202627	7	10248	7383	13113	15	11	
17	202626	7	13439	9941	16937	20	15	
18	202625	7	14105	10437	17773	21	16	
19	202624	7	8898	6016	11780	13	9	
20	202623	7	9658	6859	12457	14	10	
21	202622	7	12431	8837	16025	18	13	
22	202621	7	9372	6524	12220	14	10	
23	202620	7	7191	4114	10268	11	6	
24	202619	7	9608	6293	12923	14	9	
25	202618	7	8927	5444	12410	13	8	
26	202617	7	11063	7549	14577	16	11	
27	202616	7	12983	9199	16767	19	13	
28	202615	7	10309	7243	13375	15	10	
29	202614	7	9121	6015	12227	14	9	

…	…	…	…	…	…	…	…	…
1843	199126		7	17608	11304	23912	31	20
1844	199125		7	16169	10700	21638	28	18
1845	199124		7	16171	10071	22271	28	17
1846	199123		7	11947	7671	16223	21	13
1847	199122		7	15452	9953	20951	27	17
1848	199121		7	14903	8975	20831	26	16
1849	199120		7	19053	12742	25364	34	23
1850	199119		7	16739	11246	22232	29	19
1851	199118		7	21385	13882	28888	38	25
1852	199117		7	13462	8877	18047	24	16
1853	199116		7	14857	10068	19646	26	18
1854	199115		7	13975	9781	18169	25	18
1855	199114		7	12265	7684	16846	22	14
1856	199113		7	9567	6041	13093	17	11
1857	199112		7	10864	7331	14397	19	13
1858	199111		7	15574	11184	19964	27	19
1859	199110		7	16643	11372	21914	29	20
1860	199109		7	13741	8780	18702	24	15
1861	199108		7	13289	8813	17765	23	15
1862	199107		7	12337	8077	16597	22	15
1863	199106		7	10877	7013	14741	19	12
1864	199105		7	10442	6544	14340	18	11
1865	199104		7	7913	4563	11263	14	8
1866	199103		7	15387	10484	20290	27	18
1867	199102		7	16277	11046	21508	29	20
1868	199101		7	15565	10271	20859	27	18
1869	199052		7	19375	13295	25455	34	23
1870	199051		7	19080	13807	24353	34	25
1871	199050		7	11079	6660	15498	20	12
1872	199049		7	1143	0	2610	2	0

	inc100_up	geo_insee	geo_name
0	11	FR	France
1	5	FR	France
2	6	FR	France
3	5	FR	France
4	3	FR	France
5	2	FR	France
6	2	FR	France
7	2	FR	France
8	3	FR	France
9	4	FR	France
10	4	FR	France
11	9	FR	France
12	9	FR	France
13	13	FR	France

```
14            11        FR   France
15            18        FR   France
16            19        FR   France
17            25        FR   France
18            26        FR   France
19            17        FR   France
20            18        FR   France
21            23        FR   France
22            18        FR   France
23            16        FR   France
24            19        FR   France
25            18        FR   France
26            21        FR   France
27            25        FR   France
28            20        FR   France
29            19        FR   France
…          …       …      …
1843          42        FR   France
1844          38        FR   France
1845          39        FR   France
1846          29        FR   France
1847          37        FR   France
1848          36        FR   France
1849          45        FR   France
1850          39        FR   France
1851          51        FR   France
1852          32        FR   France
1853          34        FR   France
1854          32        FR   France
1855          30        FR   France
1856          23        FR   France
1857          25        FR   France
1858          35        FR   France
1859          38        FR   France
1860          33        FR   France
1861          31        FR   France
1862          29        FR   France
1863          26        FR   France
1864          25        FR   France
1865          20        FR   France
1866          36        FR   France
1867          38        FR   France
1868          36        FR   France
1869          45        FR   France
1870          43        FR   France
1871          28        FR   France
1872           5        FR   France
```

```
[1873 rows x 10 columns]
```

Y a-t-il des points manquants dans ce jeu de données ? Nous le vérifions.

[6]:
```
raw_data[raw_data.isnull().any(axis=1)]
```

[6]:
```
Empty DataFrame
Columns: [week, indicator, inc, inc_low, inc_up, inc100, inc100_low, inc100_up,
geo_insee, geo_name]
Index: []
```

Nous éliminons les éventuels points manquants (s'il y en a), ce qui n'a pas d'impact fort sur notre analyse qui est assez simple.

[7]:
```
data = raw_data.dropna().copy()
data
```

[7]:

	week	indicator	inc	inc_low	inc_up	inc100	inc100_low	\
0	202643	7	4736	2131	7341	7	3	
1	202642	7	2238	886	3590	3	1	
2	202641	7	2590	1247	3933	4	2	
3	202640	7	1918	812	3024	3	1	
4	202639	7	1224	234	2214	2	1	
5	202638	7	592	0	1329	1	0	
6	202637	7	862	43	1681	1	0	
7	202636	7	975	167	1783	1	0	
8	202635	7	1130	186	2074	2	1	
9	202634	7	1682	301	3063	2	0	
10	202633	7	1665	392	2938	2	0	
11	202632	7	4231	2124	6338	6	3	
12	202631	7	4064	1915	6213	6	3	
13	202630	7	6248	3767	8729	9	5	
14	202629	7	5719	3384	8054	8	5	
15	202628	7	8445	5252	11638	13	8	
16	202627	7	10248	7383	13113	15	11	
17	202626	7	13439	9941	16937	20	15	
18	202625	7	14105	10437	17773	21	16	
19	202624	7	8898	6016	11780	13	9	
20	202623	7	9658	6859	12457	14	10	
21	202622	7	12431	8837	16025	18	13	
22	202621	7	9372	6524	12220	14	10	
23	202620	7	7191	4114	10268	11	6	
24	202619	7	9608	6293	12923	14	9	
25	202618	7	8927	5444	12410	13	8	
26	202617	7	11063	7549	14577	16	11	
27	202616	7	12983	9199	16767	19	13	
28	202615	7	10309	7243	13375	15	10	

29	202614		7	9121	6015	12227	14	9
…	…	…	…	…	…	…	…	
1843	199126		7	17608	11304	23912	31	20
1844	199125		7	16169	10700	21638	28	18
1845	199124		7	16171	10071	22271	28	17
1846	199123		7	11947	7671	16223	21	13
1847	199122		7	15452	9953	20951	27	17
1848	199121		7	14903	8975	20831	26	16
1849	199120		7	19053	12742	25364	34	23
1850	199119		7	16739	11246	22232	29	19
1851	199118		7	21385	13882	28888	38	25
1852	199117		7	13462	8877	18047	24	16
1853	199116		7	14857	10068	19646	26	18
1854	199115		7	13975	9781	18169	25	18
1855	199114		7	12265	7684	16846	22	14
1856	199113		7	9567	6041	13093	17	11
1857	199112		7	10864	7331	14397	19	13
1858	199111		7	15574	11184	19964	27	19
1859	199110		7	16643	11372	21914	29	20
1860	199109		7	13741	8780	18702	24	15
1861	199108		7	13289	8813	17765	23	15
1862	199107		7	12337	8077	16597	22	15
1863	199106		7	10877	7013	14741	19	12
1864	199105		7	10442	6544	14340	18	11
1865	199104		7	7913	4563	11263	14	8
1866	199103		7	15387	10484	20290	27	18
1867	199102		7	16277	11046	21508	29	20
1868	199101		7	15565	10271	20859	27	18
1869	199052		7	19375	13295	25455	34	23
1870	199051		7	19080	13807	24353	34	25
1871	199050		7	11079	6660	15498	20	12
1872	199049		7	1143	0	2610	2	0

	inc100_up	geo_insee	geo_name
0	11	FR	France
1	5	FR	France
2	6	FR	France
3	5	FR	France
4	3	FR	France
5	2	FR	France
6	2	FR	France
7	2	FR	France
8	3	FR	France
9	4	FR	France
10	4	FR	France
11	9	FR	France
12	9	FR	France

13	13	FR	France
14	11	FR	France
15	18	FR	France
16	19	FR	France
17	25	FR	France
18	26	FR	France
19	17	FR	France
20	18	FR	France
21	23	FR	France
22	18	FR	France
23	16	FR	France
24	19	FR	France
25	18	FR	France
26	21	FR	France
27	25	FR	France
28	20	FR	France
29	19	FR	France
…	…	…	…
1843	42	FR	France
1844	38	FR	France
1845	39	FR	France
1846	29	FR	France
1847	37	FR	France
1848	36	FR	France
1849	45	FR	France
1850	39	FR	France
1851	51	FR	France
1852	32	FR	France
1853	34	FR	France
1854	32	FR	France
1855	30	FR	France
1856	23	FR	France
1857	25	FR	France
1858	35	FR	France
1859	38	FR	France
1860	33	FR	France
1861	31	FR	France
1862	29	FR	France
1863	26	FR	France
1864	25	FR	France
1865	20	FR	France
1866	36	FR	France
1867	38	FR	France
1868	36	FR	France
1869	45	FR	France
1870	43	FR	France
1871	28	FR	France

```
1872          5        FR   France

[1873 rows x 10 columns]
```

Nos données utilisent la même convention inhabituelle que pour le syndrome grippal: le numéro de semaine est collé à l'année, donnant l'impression qu'il s'agit d'un nombre entier. Nous réutilisons donc la même fonction de conversion, basée sur la bibliothèque `isoweek`, pour calculer une nouvelle colonne 'period'.

[8]:
```
def convert_week(year_and_week_int):
    year_and_week_str = str(year_and_week_int)
    year = int(year_and_week_str[:4])
    week = int(year_and_week_str[4:])
    w = isoweek.Week(year, week)
    return pd.Period(w.day(0), 'W')

data['period'] = [convert_week(yw) for yw in data['week']]
```

Nous définissons les périodes d'observation comme index de notre jeu de données, puis nous trions les points dans le sens chronologique.

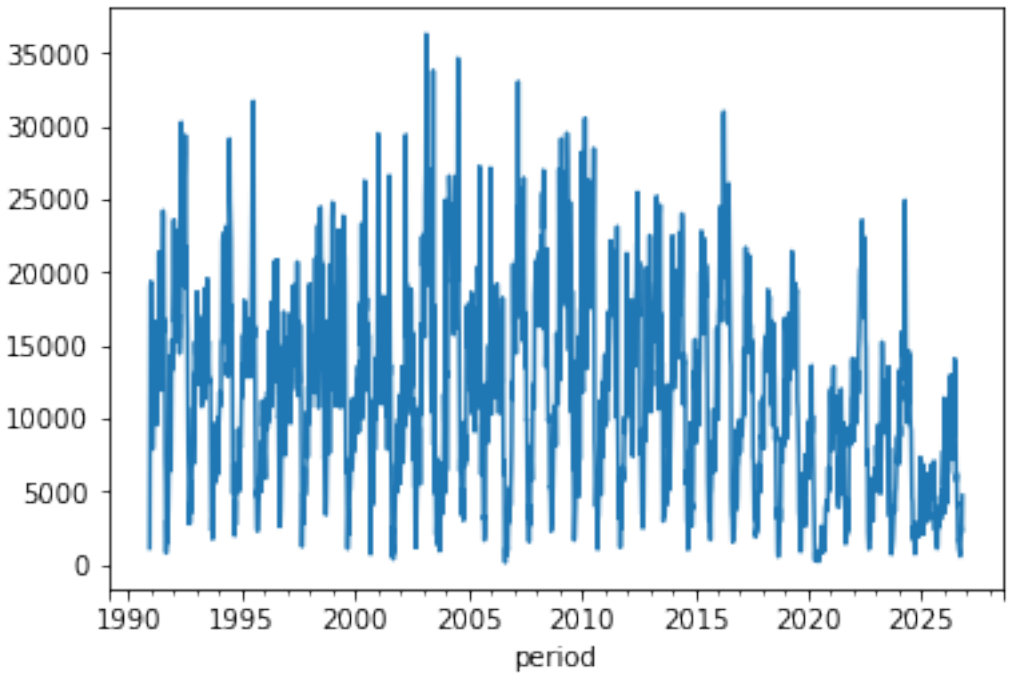
[9]:
```
sorted_data = data.set_index('period').sort_index()
```

Nous vérifions la cohérence des données: entre la fin d'une période et le début de la période qui suit, la différence temporelle doit être nulle, ou au moins très faible (marge d'erreur d'une seconde). Toute semaine absente sera affichée.

[10]:
```
periods = sorted_data.index
for p1, p2 in zip(periods[:-1], periods[1:]):
    delta = p2.to_timestamp() - p1.end_time
    if delta > pd.Timedelta('1s'):
        print(p1, p2)
```

[11]:
```
sorted_data['inc'].plot()
```

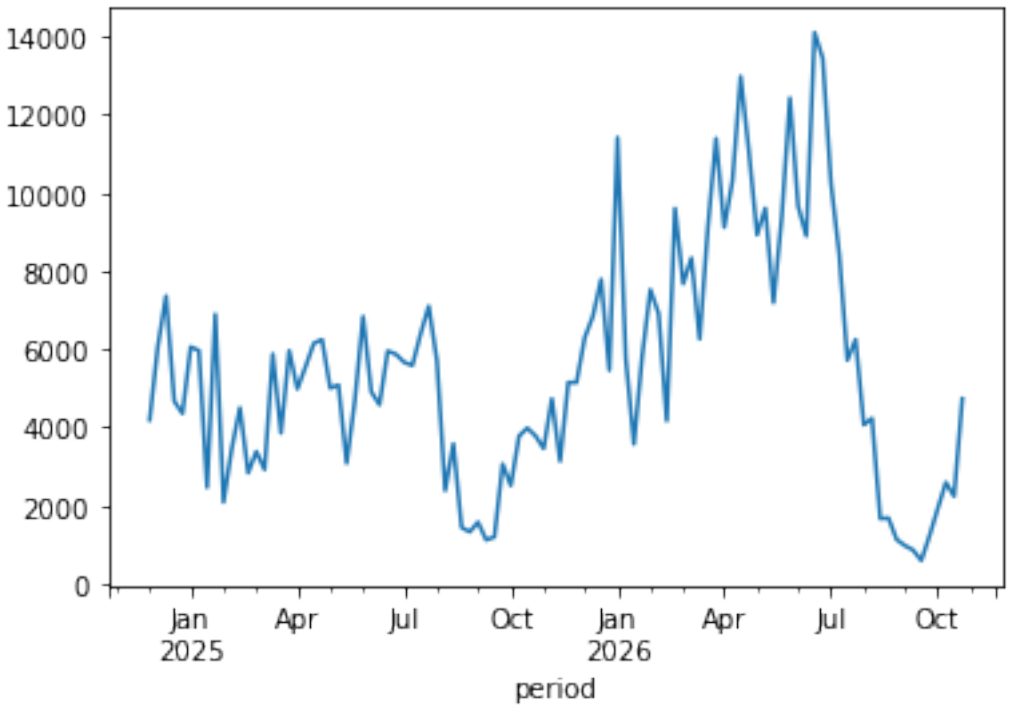
[11]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7d2e1cbe12b0>
```



Un zoom sur les dernières années montre mieux la saisonnalité. Contrairement au syndrome grippal, dont les pics sont en hiver, les pics de varicelle se situent au printemps et le creux se trouve en été.

[14]:
```
sorted_data['inc'][-100:].plot()
```

[14]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7d2e1a877e10>
```



0.1 Etude de l'incidence annuelle

Le pic de l'épidémie de varicelle se situe au printemps, donc **à l'intérieur** d'une année civile, contrairement au syndrome grippal. Mais les données commencent et se terminent en cours d'année, et le creux d'incidence se situe en été. Nous définissons donc la période de référence entre deux minima de l'incidence, du 1er septembre de l'année N au 1er septembre de l'année $N + 1$ (et non plus du 1er août).

Comme l'année ne comporte pas un nombre entier de semaines, à la place du 1er septembre de chaque année nous utilisons le premier jour de la semaine qui contient le 1er septembre. L'incidence étant très faible en été, cette modification ne risque pas de fausser nos conclusions.

Les données ne commencent pas au début d'une période: nous ne retenons que les périodes complètes. Le point de départ est calculé à partir des données, plutôt que fixé à la main comme pour le syndrome grippal, parce que cette série ne commence pas la même année. Nous nous arrêtons à la dernière période terminée.

[15]:
```
first_week = sorted_data.index[0]
last_week = sorted_data.index[-1]

first_september_week = []
for y in range(first_week.year, last_week.year + 1):
    p = pd.Period(pd.Timestamp(y, 9, 1), 'W')
```

```
    # on garde les semaines du 1er septembre qui ouvrent ou ferment une période␣
↪complète
    if p >= first_week and p <= last_week + 1:
        first_september_week.append(p)

print(first_september_week[0], '...', first_september_week[-1])
```

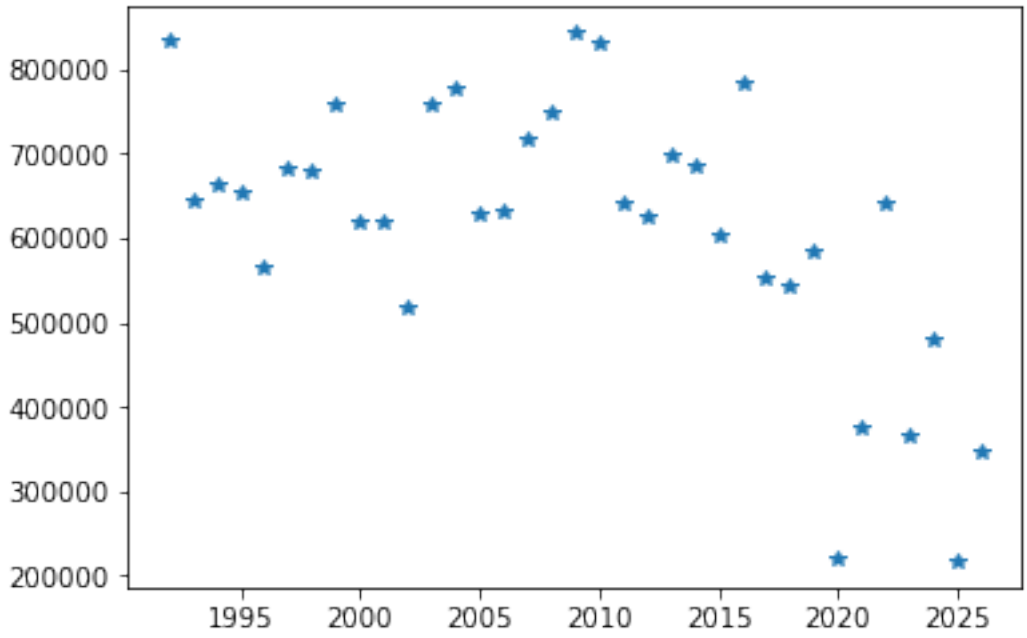
```
1991-08-26/1991-09-01 … 2026-08-31/2026-09-06
```

[16]:
```
year = []
yearly_incidence = []
for week1, week2 in zip(first_september_week[:-1],
                        first_september_week[1:]):
    one_year = sorted_data['inc'][week1:week2-1]
    assert abs(len(one_year)-52) < 2
    yearly_incidence.append(one_year.sum())
    year.append(week2.year)
yearly_incidence = pd.Series(data=yearly_incidence, index=year)
```

Voici les incidences annuelles

[17]:
```
yearly_incidence.plot(style='*')
```

[17]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7d2e1a929780>
```



Une liste triée permet de plus facilement repérer les valeurs les plus élevées (à la fin) et les plus

faibles (au début).

[18]:
```
yearly_incidence.sort_values()
```

[18]:
```
2025    217405
2020    221186
2026    347123
2023    366227
2021    376290
2024    479258
2002    516689
2018    542312
2017    551041
1996    564901
2019    584066
2015    604382
2000    617597
2001    619041
2012    624573
2005    628464
2006    632833
2022    641397
2011    642368
1993    643387
1995    652478
1994    661409
1998    677775
1997    683434
2014    685769
2013    698332
2007    717352
2008    749478
1999    756456
2003    758363
2004    777388
2016    782114
2010    829911
1992    832939
2009    842373
dtype: int64
```

[ ]:

[ ]: